POSIX Shell Implementation in C++ (mysh)

Course: Advanced Operating Systems | **Project:** Lightweight Unix Shell (`mysh`) | **Language:** C++17

1. Executive Overview & Scope

This project implements a complete, modular, and standard-compliant POSIX shell named **mysh** in C++. The shell is architected to replicate standard Linux command-line capabilities, process control mechanisms, I/O manipulation, stream pipelining, signal handling, and interactive user convenience features such as command history and tab auto-completion. Every component has been verified with comprehensive unit and integration test suites.

Key Deliverables & Verification Matrix

Feature Component	Implementation Summary	Status
Shell Prompt	Displays $ with ~ home directory substitution	**Verified**
Command Parsing	Semicolons (;) splitting with quote ('...', "...") & escape (\) support	**Verified**
Built-in: cd	Supports cd, cd .., cd ~, cd -, and arbitrary paths with OLDPWD	**Verified**
Built-in: pwd & echo	Current directory printing and space-trimmed text rendering	**Verified**
Built-in: ls (Myls)	Full custom ls with -a, -l, -la, permissions, symlinks, block counts	**Verified**
Built-in: pinfo	Extracts state, virtual memory, and exe link from Linux /proc filesystem	**Verified**
Built-in: search	Recursive DFS traversal returning True/False for target files/folders	**Verified**
Built-in: history	Maintains last 20 commands, disk persistence in mysh_history.txt	**Verified**
Process Mgmt (&)	Background execution, setpgid() detachment, async status reaping	**Verified**
I/O Redirection	Supports < (input), > (overwrite), >> (append) with dup2()	**Verified**
Pipelines (|)	Arbitrary N-stage pipelines chained via pipe() with stage redirection	**Verified**
Signal Handling	Ctrl-C (SIGINT) & Ctrl-Z (SIGTSTP) affect only foreground; Ctrl-D exits	**Verified**
Tab Completion	GNU Readline autocomplete for commands and filesystem arguments	**Verified**

2. System Architecture & Modular File Breakdown

The project follows a clean, decoupled design pattern where each major responsibility is encapsulated into its own translation unit (.cpp file) linked via the global header fun.h.

File / Module	Core Responsibility & Architecture Role
main.cpp	Shell entry point. Manages the REPL loop, prompt rendering, signal handler installation, and asynchronous background process reaping before prompt display.
fun.h	Central header declaring shared data structures (BgProcess), extern global state (fg_pid, SHELL_HOME, PREV_DIR, bg_processes), and all function signatures.
parser.cpp	Contains string manipulation routines: handleSpace() for whitespace trimming, splitCommands() for semicolon tokenization respecting quotes, and tokenizeArgs().
builtin.cpp	Dispatches built-in commands (is_builtin(), execute_builtin()). Implements cd (handling OLDPWD, tilde, and relative paths), pwd, echo, and exit.

ls.cpp	Implements Myls() mimicking GNU ls. Formats file permissions (rwxr-xr-x), hardlinks, owner/group lookup (getpwuid/getgrgid), file sizes, modification timestamps, and symlink targets.
fg_bg.cpp	Handles external program execution (executeExternal()) and background job management (CheckFgBg()). Manages process groups with setpgid() and reaps dead children.
pinfo.cpp	Parses /proc//stat and /proc//exe to extract PID, execution status (R, S, Z with '+' for foreground), virtual memory consumption, and binary path.
search.cpp	Recursive filesystem search starting from current working directory. Traverses subdirectories via opendir/readdir and returns True/False.
io_redirection.cpp	Detects redirection symbols (<, >, >>) and applies dup2() for file descriptors. Executes commands in child process ensuring parent shell isolation.
pipeline.cpp	Orchestrates multi-stage pipelines (RunPipeline()). Creates pipe file descriptors, chains child stdout to next child stdin, closes descriptors, and waits for completion.
autocomplete.cpp	Configures GNU Readline hooks (rl_attempted_completion_function). Suggests commands and executables on first token, and filesystem paths on arguments.
history.cpp	Implements command history persistence (LoadHistory(), SaveHistory(), AddHistory(), ShowHistory()). Caches up to 20 commands in mysh_history.txt without duplicates.
Makefile	Build automation file with strict flags (-Wall -Wextra -std=c++17 -lreadline) ensuring clean, reproducible builds.
test_shell.sh	Automated bash test script covering all 10 feature areas with automated assertions.

3. Technical Deep Dive: Mechanics & Inner Workings

3.1 REPL Loop & Dynamic Prompt Rendering

• **Initialization:** The shell begins by obtaining its startup directory via getcwd() and assigning it to SHELL_HOME. Signal handlers for SIGINT and SIGTSTP are configured, GNU Readline autocomplete is hooked, and past history is loaded from disk.
• **Prompt Generation:** The prompt is dynamically computed on each iteration as <username@hostname:cwd>$. The username is looked up via getpwuid(getuid()), hostname via gethostname(), and the current path is formatted using formatPathWithTilde() where paths starting with SHELL_HOME are replaced with ~.
• **Asynchronous Reaping:** Before displaying the prompt, checkBackgroundProcesses() calls waitpid(-1, &status;, WNOHANG | WUNTRACED) to reap any terminated background jobs and output their termination status.

3.2 Parsing, Quoting Rules, and Semicolons

• **Semicolon Splitting:** Input lines containing multiple commands separated by ; are split by splitCommands(). The parser tracks quote states (in_single_quote and in_double_quote) as well as escape backslashes (\), ensuring that semicolons inside quotes (e.g. echo "hello; world") are not treated as separators.
• **Argument Tokenization:** tokenizeArgs() processes each command into an array of string tokens while stripping quote delimiters and preserving embedded spaces.

3.3 Built-in Commands Implementation

• **cd:** Modifies current working directory using chdir(). Supports cd or cd ~ (jumping to SHELL_HOME), cd .. (parent), cd . (current), and cd - (jumping to PREV_DIR and echoing the resolved directory).
• **pwd:** Fetches and prints the exact current directory via getcwd().
• **echo:** Prints all space-delimited argument tokens to standard output.
• **ls (Myls):** Iterates over directories using opendir() / readdir(). When -l is provided, it invokes lstat() to extract st_mode (formatted into 10-character permissions e.g. -rw-r--r--), hardlink count, owner (via getpwuid), group (via getgrgid), byte size, formatted timestamp, and symlink targets (via readlink()). It calculates 1K filesystem block totals by summing st_blocks / 2.
• **pinfo:** Reads /proc/<pid>/stat to retrieve process state (e.g. R, S, Z, T) and Virtual Memory (vsize). Appends + if the process is in the foreground process group (pgrp == tpgid). It resolves the executable path via readlink("/proc/<pid>/exe").
• **search:** Recursively traverses subdirectories using depth-first search. Returns True if the target file/directory exists or False otherwise.
• **history:** Maintains a deque of the last 20 unique commands in memory and synchronizes with mysh_history.txt on disk.

3.4 Foreground and Background Execution Engine

• **Forking:** External commands are executed by creating a child process via fork().
• **Background Process Group Isolation:** When a command ends with &, the child process detaches from the terminal's process group by calling setpgid(0, 0). This prevents background processes from receiving terminal interrupt signals (like Ctrl-C). The parent stores the PID in bg_processes and displays [job_id] PID.
• **Foreground Process Tracking:** For foreground processes, the parent assigns fg_pid = pid and blocks using waitpid(pid, &status;, WUNTRACED). If the child is suspended via Ctrl-Z, it is moved to the background list.

3.5 I/O Redirection & Multi-Stage Pipelines

• **I/O Redirection:** Handled by apply_redirection(). Opens input files with O_RDONLY and replaces stdin via dup2(fd_in, STDIN_FILENO). Output files are opened with O_WRONLY | O_CREAT | O_TRUNC (for >) or O_APPEND (for >>) and redirected via dup2(fd_out, STDOUT_FILENO).
• **Pipeline Orchestration:** Multi-stage pipelines (cmd1 | cmd2 | ... | cmdN) create $N-1$ pipes via pipe(). For child i, dup2() attaches stdin to the read end of pipe $i-1$ (if $i > 0$) and stdout to the write end of pipe i (if $i < N-1$). All unused pipe file descriptors are closed across all children and the parent process to prevent deadlock and ensure proper EOF propagation.
• **Combined Features:** Pipelines fully support integrated I/O redirection on individual stages (e.g. cat < in.txt | grep text > out.txt).

3.6 Signal Handling & GNU Readline Integration

• **SIGINT (Ctrl-C):** If a foreground process is running (fg_pid > 0), kill(fg_pid, SIGINT) forwards the interrupt to terminate the child without affecting the shell. If idle at the prompt, Readline moves to a fresh new line.
• **SIGTSTP (Ctrl-Z):** Forwards SIGTSTP to fg_pid, suspending the child process and pushing it into the background job table.
• **EOF (Ctrl-D):** Detected when readline() returns nullptr. The shell gracefully saves history and terminates.
• **Tab Completion:** Hooks rl_attempted_completion_function. Suggests built-in commands and local directory binaries on the first token, and filesystem paths for arguments.

4. POSIX System Calls Reference Table

System Call	Category	Specific Purpose in mysh
fork()	Process	Spawns child processes for external commands, redirection, and pipeline stages.
execvp()	Process	Replaces child process image with the specified executable binary.
waitpid()	Process	Waits for foreground processes or checks background status non-blockingly (WNOHANG).
setpgid()	Process	Sets process group ID to isolate background processes from terminal signals.
pipe()	IPC	Creates unidirectional data channels connecting pipeline stages.
dup2()	I/O	Duplicates file descriptors to redirect stdin/stdout for pipes and files.
open() / close()	I/O	Opens file handles for redirection with O_RDONLY, O_WRONLY, O_CREAT, O_APPEND, O_TRUNC.
chdir() / getcwd()	Filesystem	Navigates directories in cd and resolves current working directory in pwd and prompt.
opendir() / readdir()	Filesystem	Iterates through directory entries in custom ls and recursive search.
lstat() / stat()	Filesystem	Queries file metadata (permissions, sizes, timestamps, blocks, file types).
readlink()	Filesystem	Reads target of symbolic links in ls -l and resolves /proc//exe in pinfo.
sigaction() / kill()	Signals	Installs handlers for SIGINT, SIGTSTP and forwards signals to foreground process.

5. Automated Test Suite (test_shell.sh)

The project includes a robust automated validation script (test_shell.sh) verifying all functional requirements:

- **Test 1: Basic Built-ins:** Validates pwd, echo, cd .., cd -, and clean exit.
- **Test 2: Semicolon Sequences:** Executes multiple commands chained via ; sequentially.
- **Test 3: I/O Redirection:** Tests file output creation (>), input reading (<), and appending (>>).
- **Test 4: Multi-stage Pipelines:** Validates multi-process chaining: ls -l | grep -E '\.cpp|\.h' | wc -l.
- **Test 5: Pipelines with Redirection:** Tests integrated input/output redirection within pipeline stages.
- **Test 6: Search Command:** Validates recursive file search returning True for existing files and False otherwise.
- **Test 7: Process Info (pinfo):** Tests /proc inspection for PID, state (R+), memory, and executable path.
- **Test 8: Custom ls Listing:** Tests ls -a, ls -l, and combined ls -la with full permissions & details.
- **Test 9: Background Processes:** Spawns background sleep 1 &, executes foreground task, and verifies async reaping.
- **Test 10: Command History:** Verifies history retrieval and disk persistence with history 5.

6. Compilation and Execution Guide

Compilation:
$ make clean && make

Running the Shell:
$ ./mysh

Running Automated Test Suite:
$ chmod +x test_shell.sh && ./test_shell.sh